Computational Data Analysis (02582)

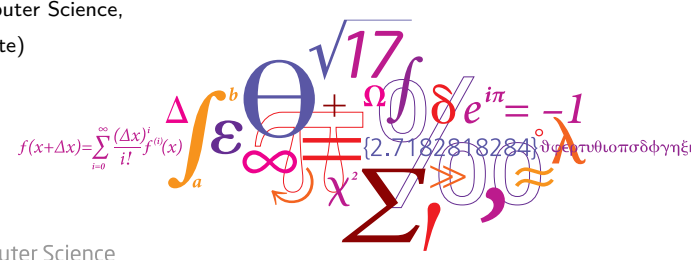
Classification and Regression Trees (CART) and bagging

Line Clemmensen

Professor

Department of Applied Mathematics and Computer Science,

Technical University of Denmark (DTU Compute)



DTU Compute

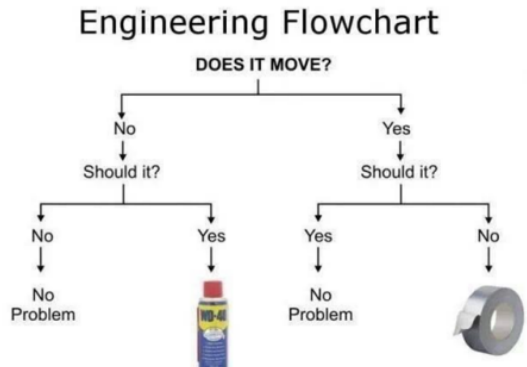
Department of Applied Mathematics and Computer Science

Outline I

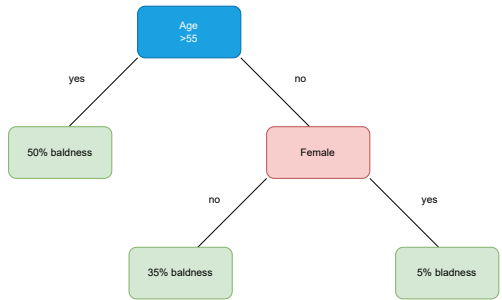
- Regression trees
- Classification
- Bagging

Classification and regression trees

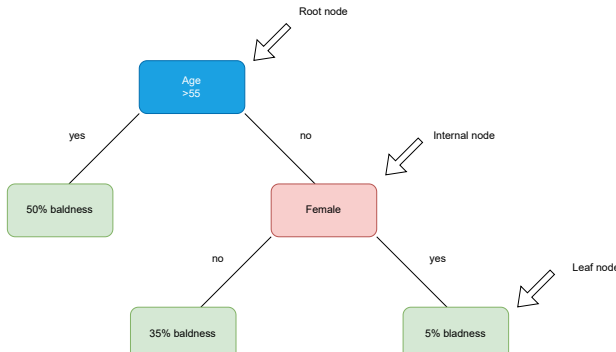
A decision tree mimics a series of decisions based on expert knowledge but is data-based rather than expert-driven.



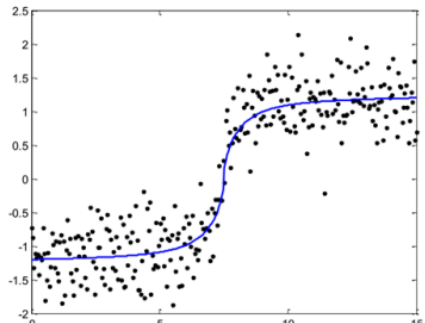
Regression Tree - concept



Regression Tree - concept



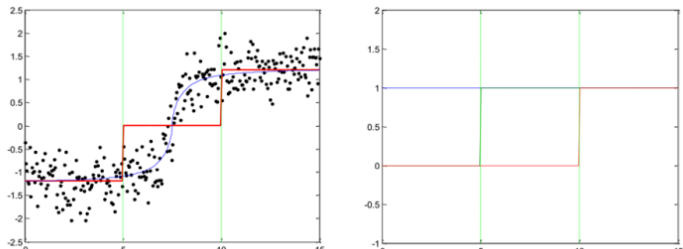
Regression Tree - example



- True function $y = f(x)$ blue
- Observations with noise: (x_i, y_i) black

Regression Tree - example

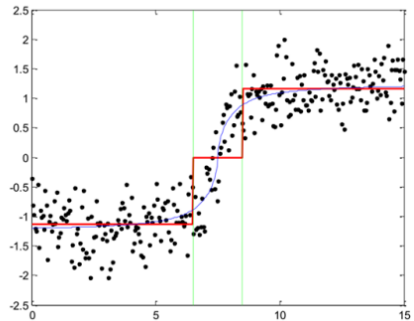
Let's try to fit a simple piecewise polynomial



- X = [ones(n,1) double(x>5) double(x>10)];
- A constant in each interval
- Evenly spaced knots

A simple example

Let's try to make a better placement for the knots



A simple example

Making our knot placement into an algorithm

- Choose a number of knots k.
- Try all possible positions for each knot
  - Infinite number of combinations

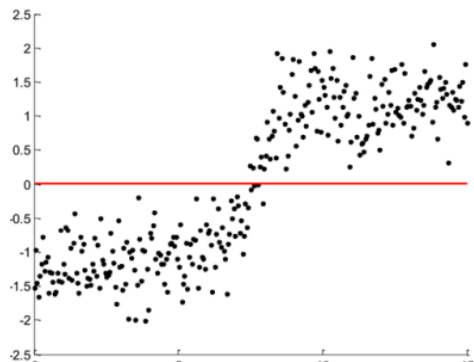
Let's try with 100 different positions for the knot placement on the x-axis

- 100^k positions to try
- With 5 knots (k=5), we get = 10.000.000.000 combinations
- Add additional variables, and your computer blows up

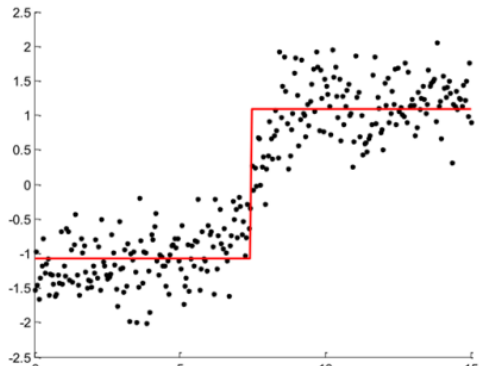
A simple example

New idea:

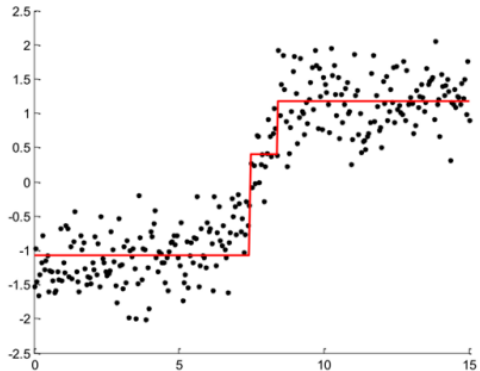
- Place the knots one after another
- Place each knot such that the fit is as good as possible



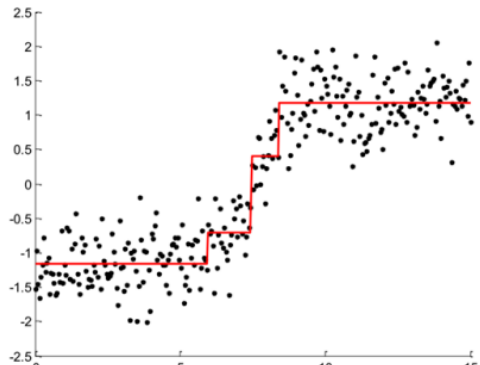
A simple example



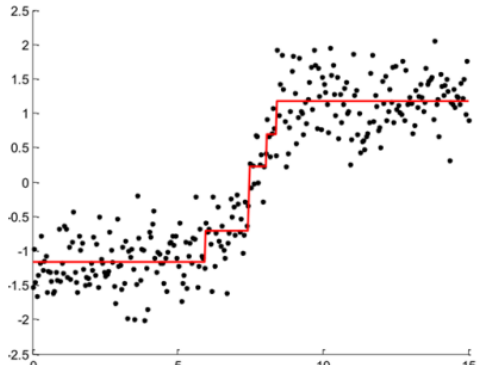
A simple example



A simple example

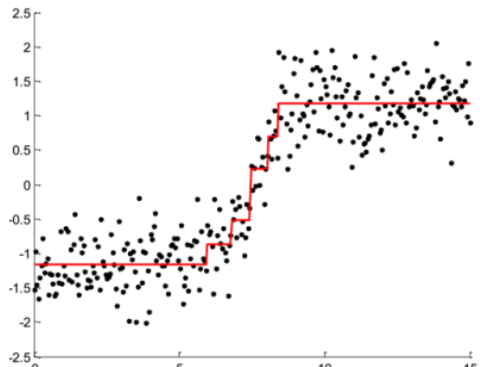


A simple example



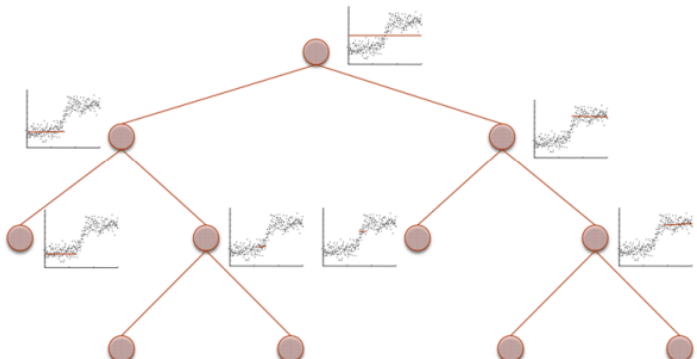
A simple example

At this point, the regression function is pretty good!



Tree representation

Each split can be represented by a parent node split into two child nodes

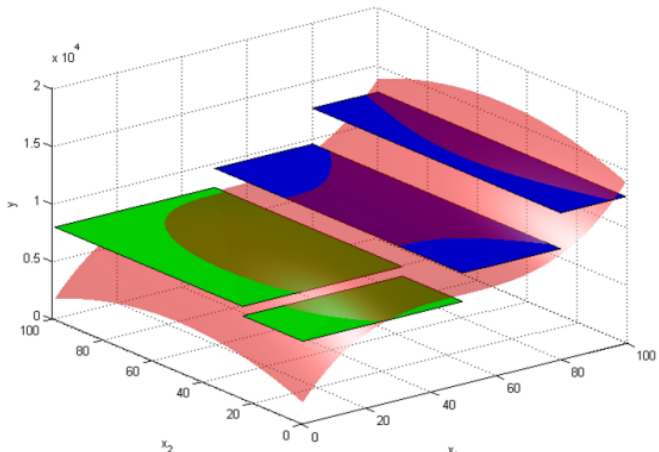


More than one input variable

Handled in the simplest way possible

```
For each input variable
    For each split
        Evaluate split point
    End
End
```

More than one input variable



Key questions

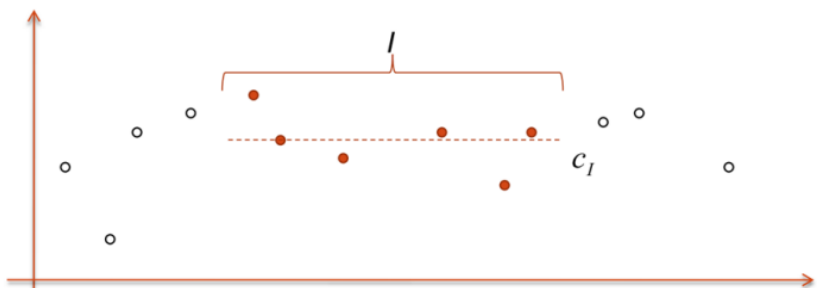
- What is a good split?
  - We need to know this to decide where to split
- How many splits should we try?
  - We used 100 before - is there a better choice?
- When do we stop splitting?

What is a good split?

- The terminal nodes each represent an interval of the input variable(s).
- We choose to represent the outcome in this interval by a constant function.
- As for most regression problems, we say that a constant function is good if it has low residual sum of squares when compared to training outcome data.

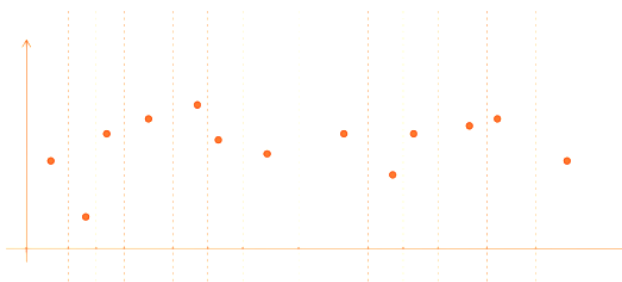
What is a good split?

- Prediction $\hat{y}$ is a constant in each interval I
- Residual sum of squares (RSS) in interval I: $RSS_I = \sum_{i \in I}(y_i - \hat{y}_i)^2$
- Minimal when $\hat{y} = \sum_{i \in I} y_i / n$



How many splits?

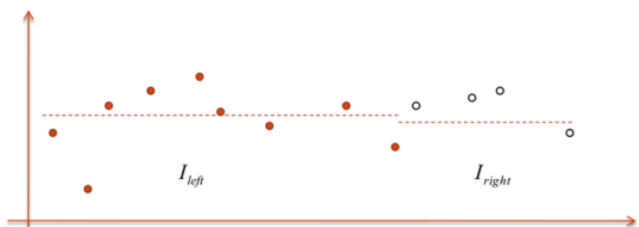
- Optimal constant function is the average of the outcome in the interval
- Only important if observations are in the interval or not
- Split somewhere in each gap between observations



How many splits?

In an interval with n_I observations we have n_I possible splits

- Nothing fancy here, just try them all
- Choose the one with the lowest total RSS on the interval



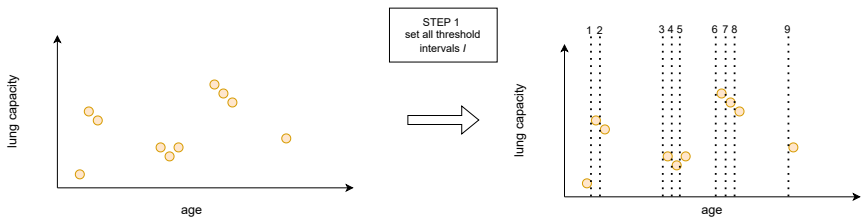
$$RSS_I = RSS_{I_{left}} + RSS_{I_{right}}$$

Splitting categorical predictors

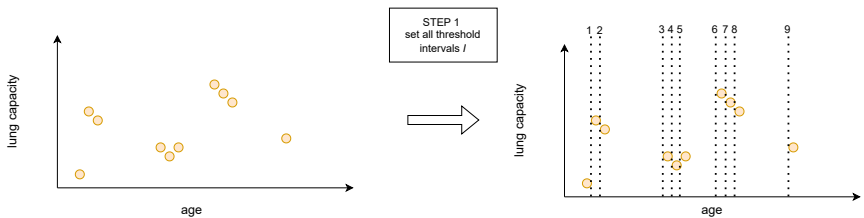
In an interval with n_l observations we have n_l possible splits

- Consider a two-category input (e.g. male-female)
  - Only one way to split, men versus women
  - Empty groups are not allowed
  - (male),(female) and (female),(male) is the same split
- Consider a three-category input (apple, orange, banana)
  - Three possible splits
    (apple) , (orange, banana)
    (apple, orange) , (banana)
    (apple, banana) , (orange)
- How many splits for a variable with k categories?
  - In the exercises of today!

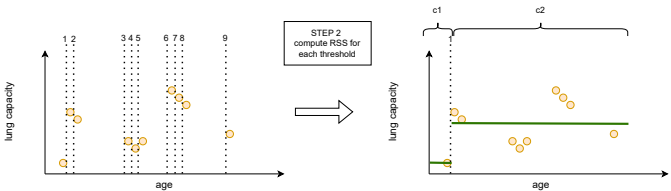
A complete wrap



A practical example



A practical example



Calculate the means (green) of each interval (c1 - left and c2 - right)

$$\hat{y}_{c1} = 10, \hat{y}_{c2} = 30$$

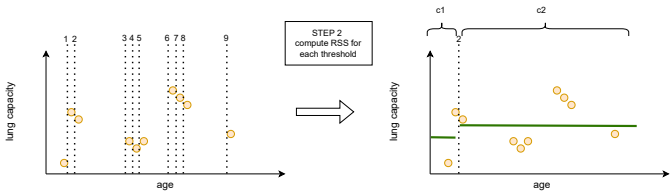
Calculate RSS for each interval (c1 - left and c2 - right):

$$RSS_{c1} = (10 - 10)^2 = 0 \quad (1$$

$$RSS_{c2} = (33 - 30)^2 + (31 - 30)^2 + (25 - 30)^2 + \cdots = 54 \quad (2$$

$$RSS_l = RSS_{c2} + RSS_{c1} = 54 \quad (3$$

A practical example



Calculate the means (green) of each interval (c1 - left and c2 - right)

$$\hat{y}_{c1} = 21.5, \hat{y}_{c2} = 29$$

Calculate RSS for each interval (c1 - left and c2 - right):

$$RSS_{c1} = (10 - 21.5)^2 + (33 - 21.5)^2 = 264.5 \quad (4$$

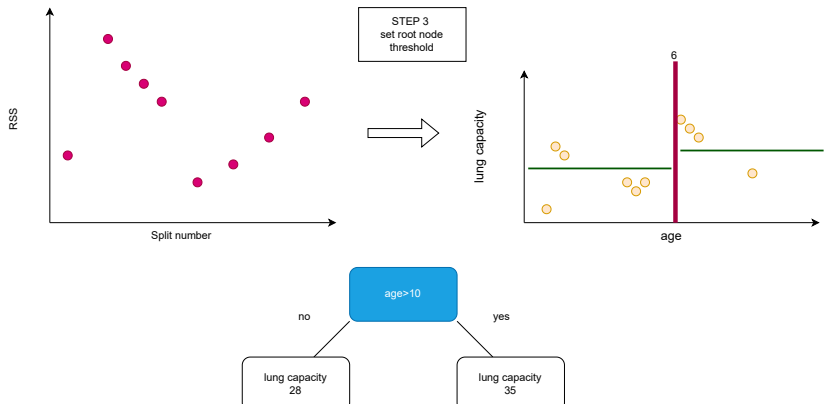
$$RSS_{c2} = (31 - 29)^2 + (25 - 29)^2 + (23 - 29)^2 + \cdots = 50 \quad (5$$

$$RSS_l = RSS_{c2} + RSS_{c1} = 314.5 \quad (6$$

A practical example

Note down the RSS_l for the l'th segment.

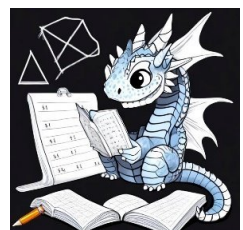
Choose the threshold with the lowest RSS - the root node is now found



It is possible to do more splits at each side of the threshold

A practical example

What do we do when we have more than one variable?



How large do we grow the tree?

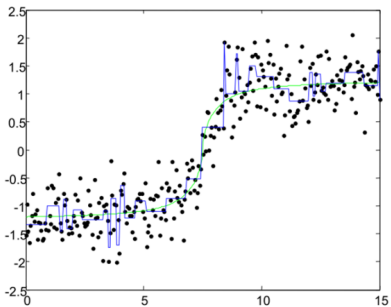
- Stop splitting when a node contains too few observations
- For instance, do not split nodes with 10 or less observations

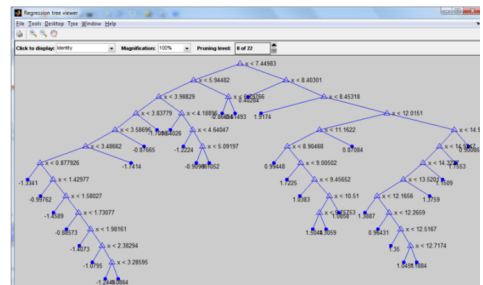
Tree-growing procedure

- First interval (node) is the entire range of X.
- For each variable
  - For each splitting position,
    calculate $RSS = RSS_{left} + RSS_{right}$
  - Remember position with the lowest RSS
- Split the variable with the lowest RSS at the corresponding position
- This produces a left and right sub-interval (child-nodes)
  - Split each of these into child nodes as above
  - Keep splitting nodes until a node contains too few observations
- Assign a constant function to terminal nodes, the average of the observations

Returning to the example

What happens when we set the splitting rule as not splitting nodes with 10 or less observations:





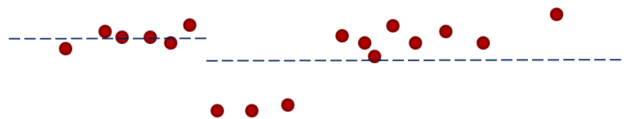
What do you think of this fit?

Finding the right sized tree

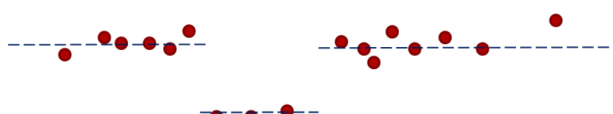
- The tree was split too far - overfitting
- How do we know when to stop splitting?
  - Answer: We do not - A seemingly worthless split may lead to excellent splits below

First split

Finding the right sized tree



Second split



Third split

Pruning Rule

- We do not want to miss out on good splits
- Strategy: Grow the tree really large (small MinParent) and then decide which splits were unnecessary and remove these.
- This is called **pruning** the tree
  - Pruning a node amounts to removing its sub-tree, thereby making a terminal node or leaf node.

Pruning Rule

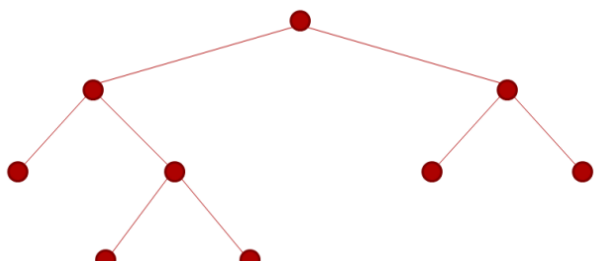
Prune the non-terminal node whose sub-tree gives the smallest **per node** reduction in RSS.

- Divide the reduction by the number of terminal nodes minus one

Pruning the tree

Weakest-link pruning: prune branches that contribute the least to lowering RSS

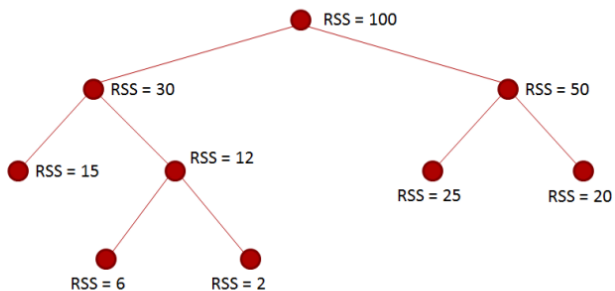
Example:



Pruning the tree

Weakest-link pruning: prune branches that contribute the least to lowering RSS

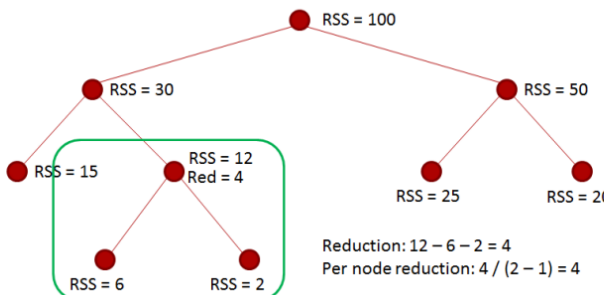
Example:



Pruning the tree

Weakest-link pruning: prune branches that contribute the least to lowering RSS

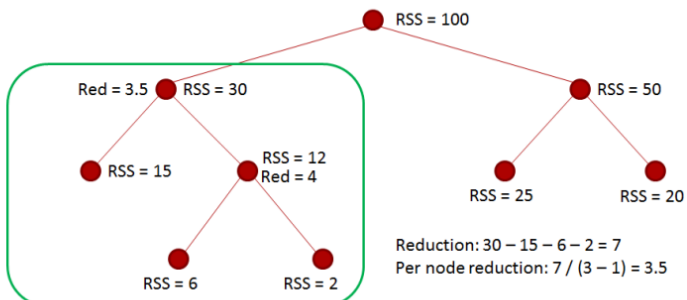
Example:



Pruning the tree

Weakest-link pruning: prune branches that contribute the least to lowering RSS

Example:

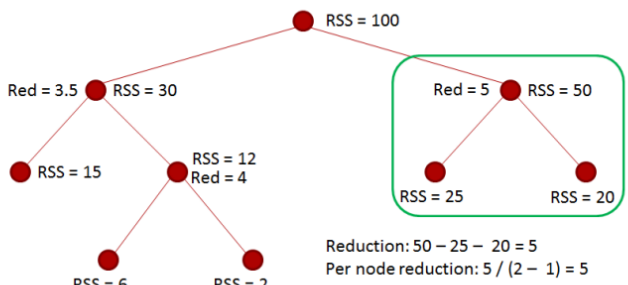


Reduction: 30 – 15 – 6 – 2 = 7
Per node reduction: 7 / (3 – 1) = 3.5

Pruning the tree

Weakest-link pruning: prune branches that contribute the least to lowering RSS

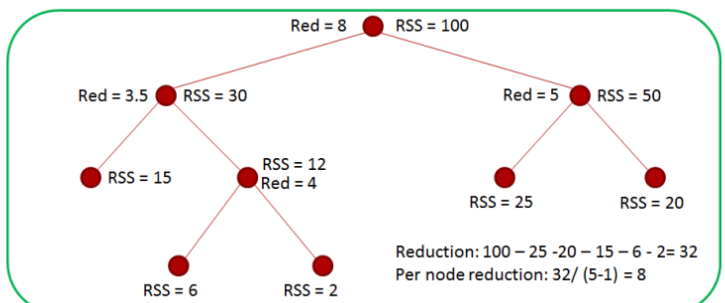
Example:



Pruning the tree

Weakest-link pruning: prune branches that contribute the least to lowering RSS

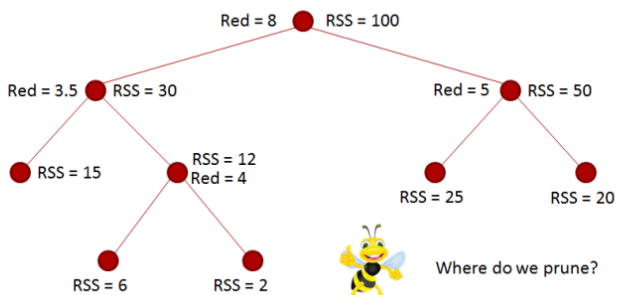
Example:



Pruning the tree

Weakest-link pruning: prune branches that contribute the least to lowering RSS

Example:



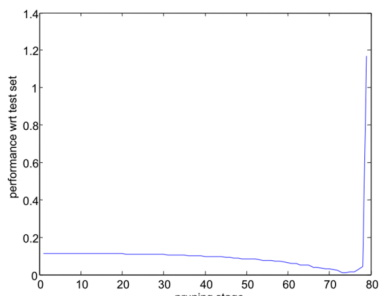
When to stop pruning?

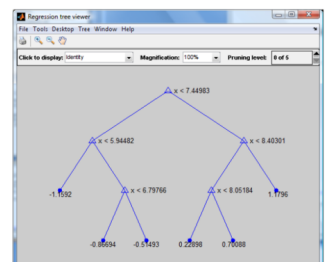
- If we keep pruning, we will end up with just the root node
- Which of all these sub-trees do we choose?

- Two approaches
  - Independent test set
  - Cross-validation

Using an independent test set

- As we prune our way towards the root node, evaluate the performance of each sub-tree with respect to the test set.
- Continue all the way to the root node
  - Choose sub-tree with best performance





Using cross-validation

Three alternatives...

- Cross validate all possible trees, does not work in practice - too many trees.
- Use a tuning parameter. Choose the tree that minimizes,

$$RSS(T) + \alpha|T|$$

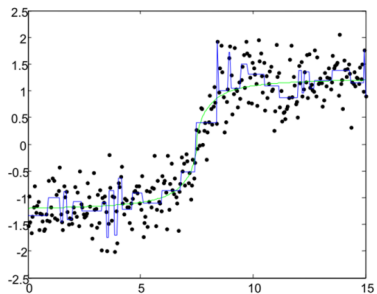
$|T|$= number of end-nodes

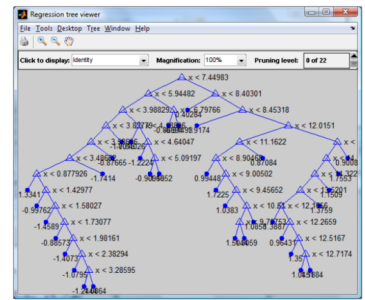
Find the tuning parameter α using cross-validation

- Use the cross-validation to determine the optimal value of the minimum number of observations in a terminal node. Matlab parameter MinLeaf. Then, use the full-grown tree without pruning.

Regression example revisited

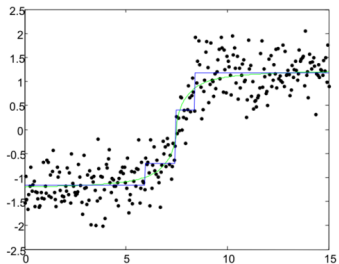
`MinParent = 20` full tree

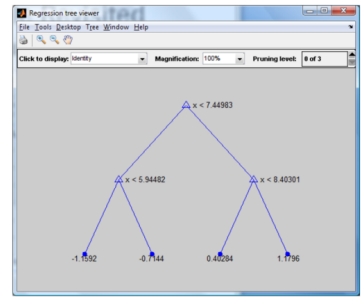




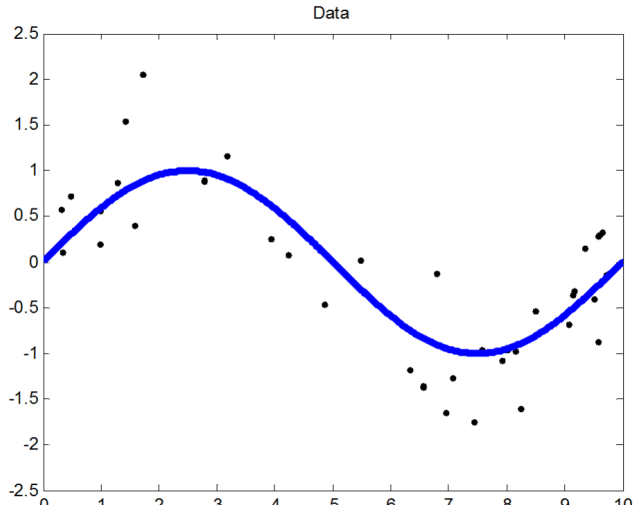
Regression example revisited

Best sub-tree chosen by 10-fold cross-validation.

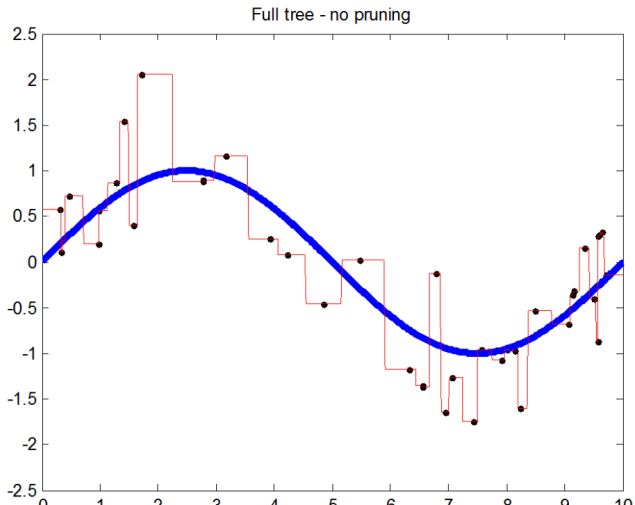




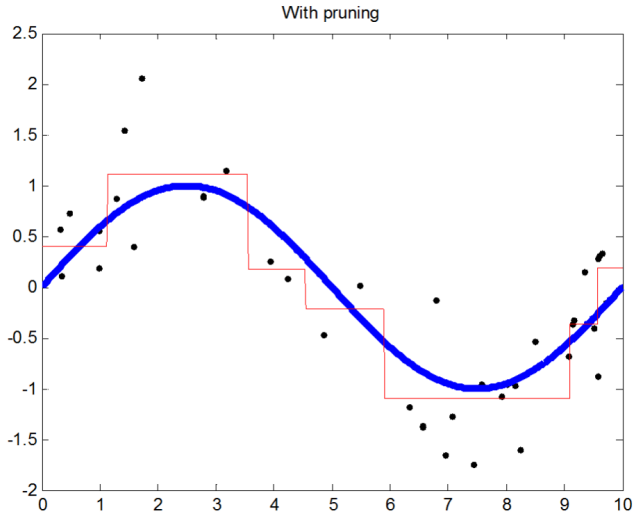
Bias and variance trade-off



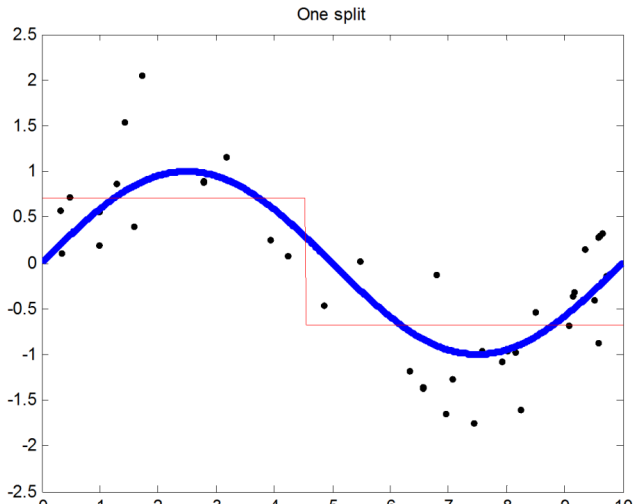
Bias and variance trade-off



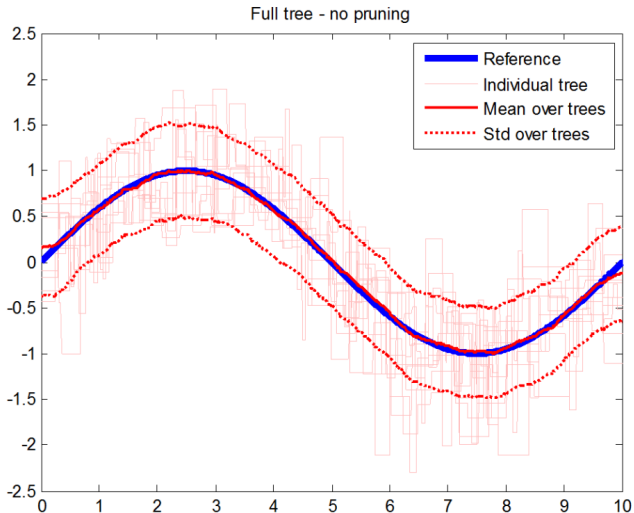
Bias and variance trade-off



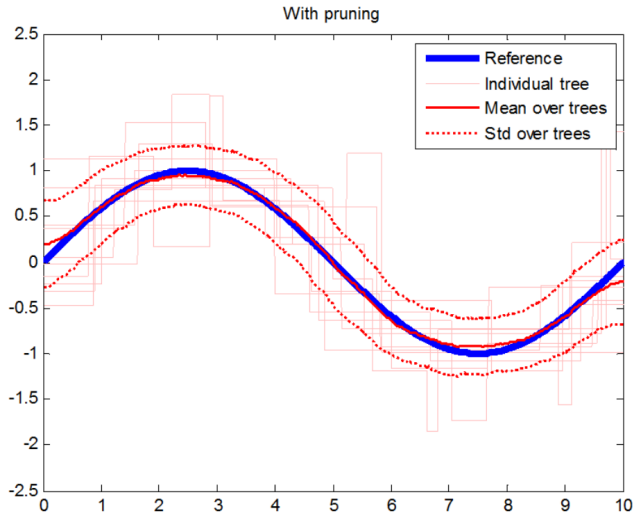
Bias and variance trade-off



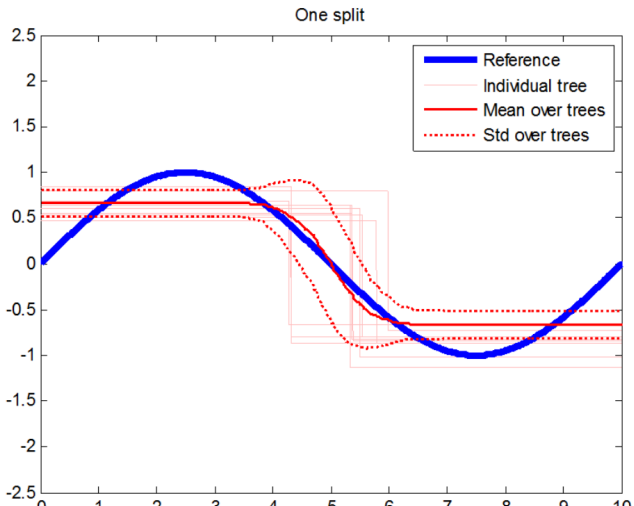
Bias and variance trade-off



Bias and variance trade-off



Bias and variance trade-off

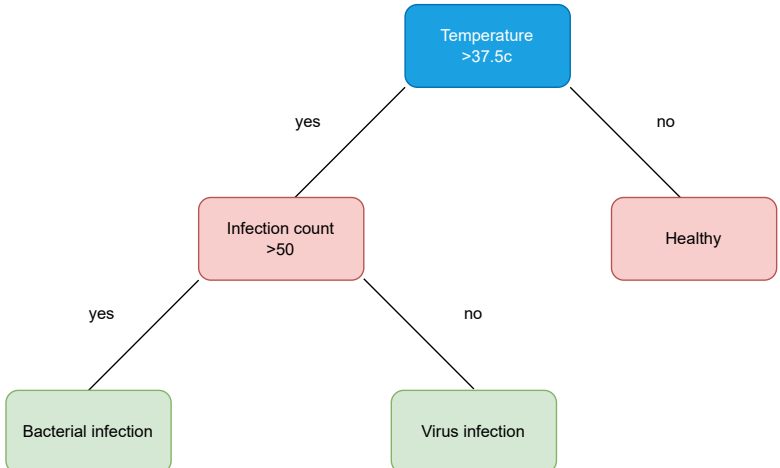


Bias and variance trade-off

What do we conclude about regression trees in terms of

- Bias
- Variance

Classification trees - concept



Classification trees

The classification tree assigns a class instead of a constant.

It has an almost identical process as regression trees, but there are different criteria for splitting nodes and pruning the tree.

Node values

With **regression trees** we transform a new observation into a constant.

- The constant was derived from the training data
- It was the **mean** of the output variable of the training observations in the node

For **classification trees**, we assign new observations to a certain class

- The class is derived from training data
- It is the **majority class** of the training observations in the node

Model errors

Regression trees

For regression trees, we used RSS as a measure of node impurity

Classifcation trees

- For classification trees, we have a few options
  - Missclassification rate
  - Gini index
  - Cross-entropy
- They all favor the split that increases purity the most
  - Typically, the predictive performance is not that different
  - The shape of the trees might, however, be very different

Node impurity for classification trees

In a specific node, representing a region R with N observations, let

$$\hat{p}_k = \frac{1}{N} \sum_{x_i \in R} \mathbb{1}(y_i = k)$$

Classify observations in the node to class,

$$K = \arg\max_k \hat{p}_k$$

Measures of impurity within a node,

Misclassification error: $Q = \frac{1}{N} \sum_{x_i \in R} \mathbb{1}(y_i \neq k) = 1 - \hat{p}_k$

Gini index: $Q = \sum_{k \neq k} \hat{p}_k \hat{p}'_k = \sum_k \hat{p}_k (1 - \hat{p}_k)$

Cross-entropy (deviance): $Q = -\sum_k \hat{p}_k log(\hat{p}_k)$

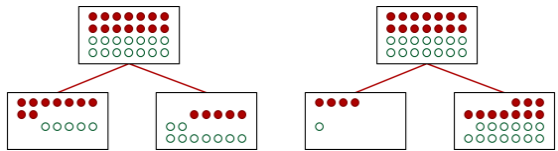
From node impurity to split criterion

- The node impurity is weighted with the number of observations in each node.
- The split decision is based on the split that **minimizes**,

$$N_{left} Q_{left} + N_{right} Q_{right}$$

- N, the number of observations in left and right node
- Q, node impurity for left and right node

Comparing splits

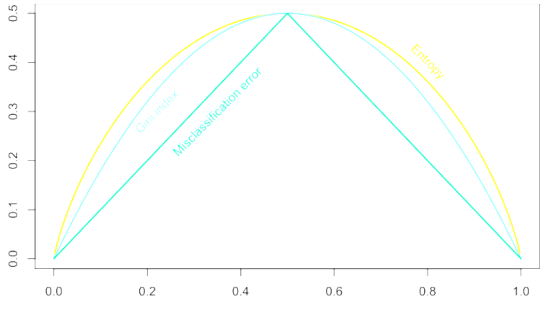


Misclassification error
left node: 5/14
right node: 5/14
Split Criterion
14(5/14)+14(5/14)=10

Misclassification error
left node: 1/5
right node:10/23
Split Criterion
5(1/5)+23(10/23)=11

The lower the impurity, the better. Now try the same with the Gini index

Node impurity for classification trees



- Node impurity measures for a two-class problem.
- X-axis: proportion of samples belonging to class 2.

- Entropy and Gini index are better measures for growing tree because they are more sensitive to node probabilities.

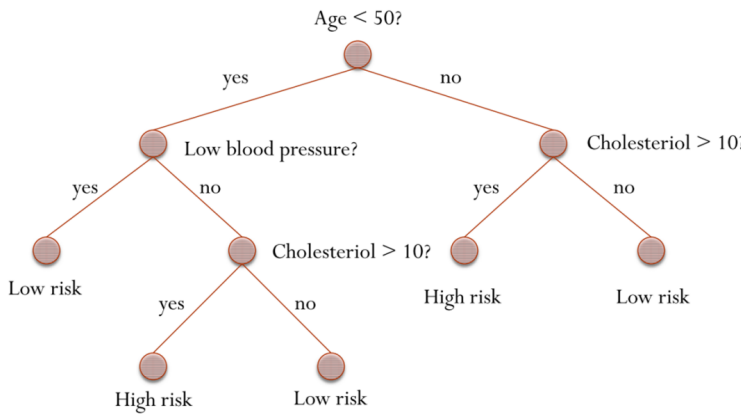
Standard practice

- Use **Gini index** as split criterion when **building** the tree.
- Use **missclassification rate** as criterion when deciding which node to **prune**.

Benefits of tree structure

Interpretability!

Consider the following (classification) tree on heart diseases



Interpretability

- CART is popular in medical sciences because it may represent the way doctors reason
- A single tree describes the entire partitioning of the input space.
- With $p > 3$ input variables, the partition (cf. knot positions) are difficult to visualize.
  - But a tree representation is always possible.
- A large tree might be difficult to interpret anyway...

Missing data

Incomplete data are common in many applications.

We can always

- Delete the observation
- Replace with mean or median

For trees, we can

- Introduce an extra category as missing - if it is a categorical variable.
- Use a surrogate variable. In each branch, have a list of alternative variables and split points as a backup.
  - Matlab does this.

Bagging

- Ensemble method
- Many models on bootstrap samples
- Output from all models aggregated into one model

Review - bootstrapping

Start with data

$$X = \begin{bmatrix} 1 & 2 \\ 2 & 4 \\ 3 & 6 \\ 4 & 8 \\ 5 & 10 \end{bmatrix}$$

sample with replacement

$$X_1' = \begin{bmatrix} 3 & 6 \\ 2 & 4 \\ 3 & 6 \\ 5 & 10 \\ 3 & 6 \end{bmatrix}, X_2' = \begin{bmatrix} 4 & 8 \\ 4 & 8 \\ 2 & 4 \\ 1 & 2 \\ 3 & 6 \end{bmatrix}, X_3' = \begin{bmatrix} 1 & 2 \\ 5 & 10 \\ 2 & 4 \\ 3 & 6 \\ 4 & 8 \end{bmatrix}$$

Bagging

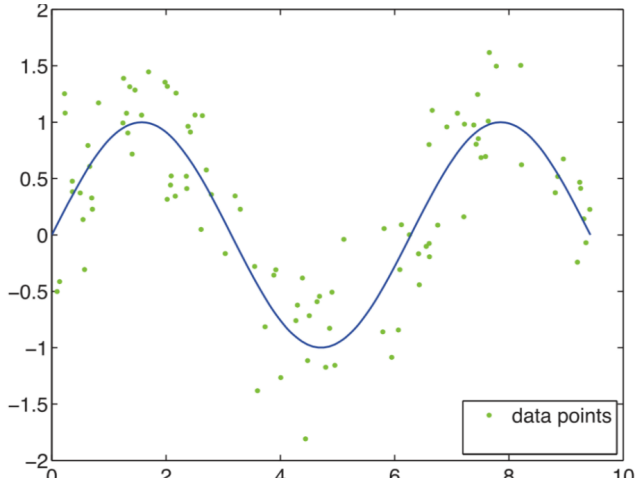
- Use bootstrap to improve prediction (not to tune parameters or assess prediction errors)
- Bagging averages predictions over a collection of bootstrap samples
- Average many noisy but approximately unbiased models and thereby reduce the variance

Bagging algorithm

- Make B boostrap samples of of size N
- For b = 1 to B repeat
  - Fit a model using the bth sample and make the prediction $\hat{y}_b$
- The bagging estimate is given by the average of the B predictions

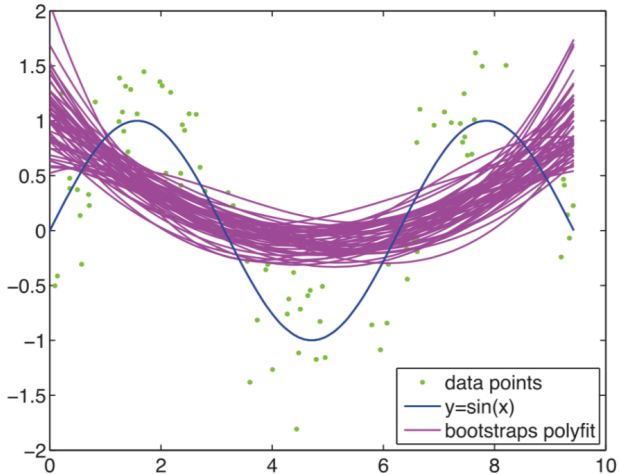
$$\hat{y}_{bagging} = \frac{1}{B} \sum_{b=1}^{B} \hat{y}_b$$

Bagging example - sin(x)



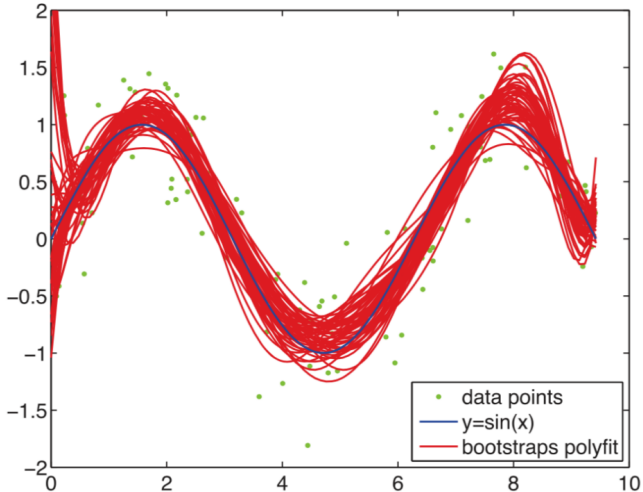
100 data points with white noise added from $y = \sin(x)$.

Bagging example - sin(x)



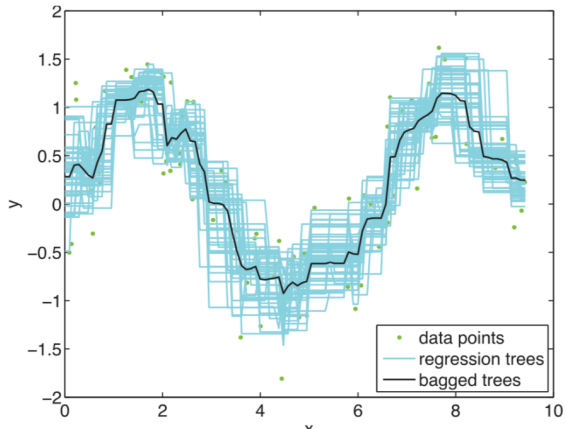
Fits of a 3rd degree polynomial to 50 bootstraps of data.

Bagging example - sin(x)



Fits of a 9th degree polynomial to 50 bootstraps of data.

Bagging example - sin(x)



Fits of regression trees to 50 bootstrap samples and the bagged tree

Bagging bias

The bias of the bagged trees is the same as that of the individual trees as the trees are identically distributed

$$E(\hat{y} - y) = E[\frac{1}{B}\sum_{b=1}^{B}(\hat{y}_b - y)] = \frac{1}{B}E[(\hat{y}_b - y)] = E[(\hat{y}_b - y)]$$

Bagging variance

The variance of the average of the bagged estimates is (ρ is the correlation between pairs of trees)

$$\rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$$

The second term goes to zero as B increases. We see that the size of the correlation between pairs of bagged trees, (ρ), is what limits the variance reduction.

Bagging summary

- Particularly good for high-variance, low-bias methods such as trees
- **Regression**
  - Regression trees are fitted to bootstrap samples of the training data. The result is the average over all of the trees.
- **Classification**
  - A committee of trees each cast a vote for the class - and the majority vote is used as the prediction.

Today's Exercises

- imdb data example.*Key learning*: **Evaluate the steps in designing and building a CART (pen and paper)**.
- Cleveland Heart Data. *Key learning*: **Use existing algorithms to build and evaluate a CART**.
- Zip data.*Key learning*: **Build a bagging model and evaluate importance of bagging hyper parameters for both individual models and the bagging procedure**.

Questions?

Any questions for today's topic?